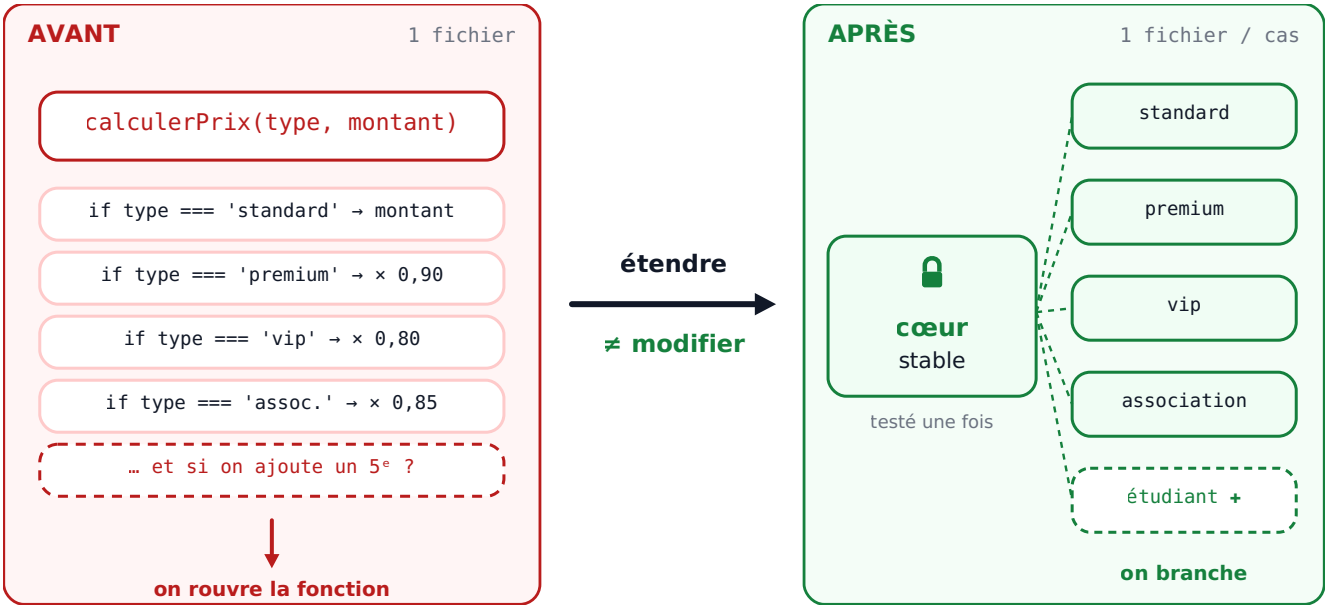
SOLID · LE O

Open /Closed

Ouvert à l'extension, fermé à la modification

Tout le principe en schémas : ce qui casse, ce qu'on coud, et les 4 seules coutures à connaître. Une page = une idée visuelle.



1	La définition, en un seul schéma	2
2	Le test des 5 minutes	3
3	Les 4 coutures : interface, registre, pipeline, données	4
4	Anatomie de l'avant · anatomie de l'après	5-6
5	8 cas réels, avant / après en images + code	7-10
6	Le reste de la boîte en un coup d'œil	11
7	Le piège : sur-abstraire · l'arbre de décision	12-13
8	Récap visuel et cartes de révision	14

> Pour réviser : les schémas seuls suffisent — le code n'est là que pour l'ancrage. Une double-page = une idée. Imprime en A4, garde les pages 4 et 14 sous les yeux.

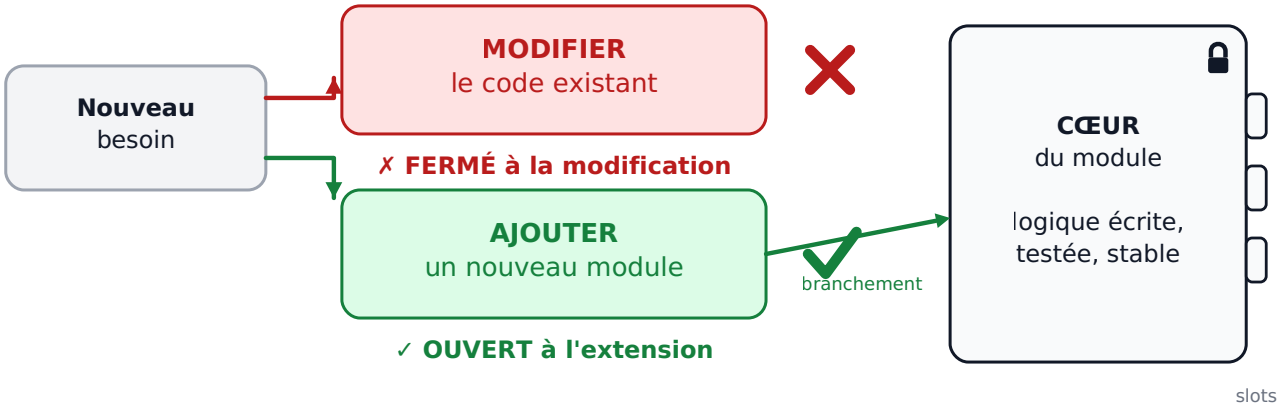
« Ajouter un cas ne doit pas rouvrir la logique qui marche. »

Fermé à la modification · ouvert à l'extension

1 · LA DÉFINITION

Ouvert à l'extension, fermé à la modification

Un module doit pouvoir changer de comportement sans qu'on touche à son code déjà écrit et déjà testé.



Comportement de base + nouveau module = nouveau comportement

et le cœur n'a pas bougé d'une ligne

1 · Ce qui est fermé

Le **cœur** : la logique métier, les décisions déjà validées par des tests.

2 · Ce qui est ouvert

Les **coutures** : points où l'on branche un module sans rien réécrire.

3 · Le mot « extension »

Ajouter une ligne de données dans un registre, c'est de l'extension. Rouvrir un `if` qui marche, non.

AVANT — fermé à l'extension

```
if (type === "vip")      return m * 0.8;
if (type === "etudiant") return m * 0.95;  // ← on ajoute ici
//    dans une fonction déjà testée
```

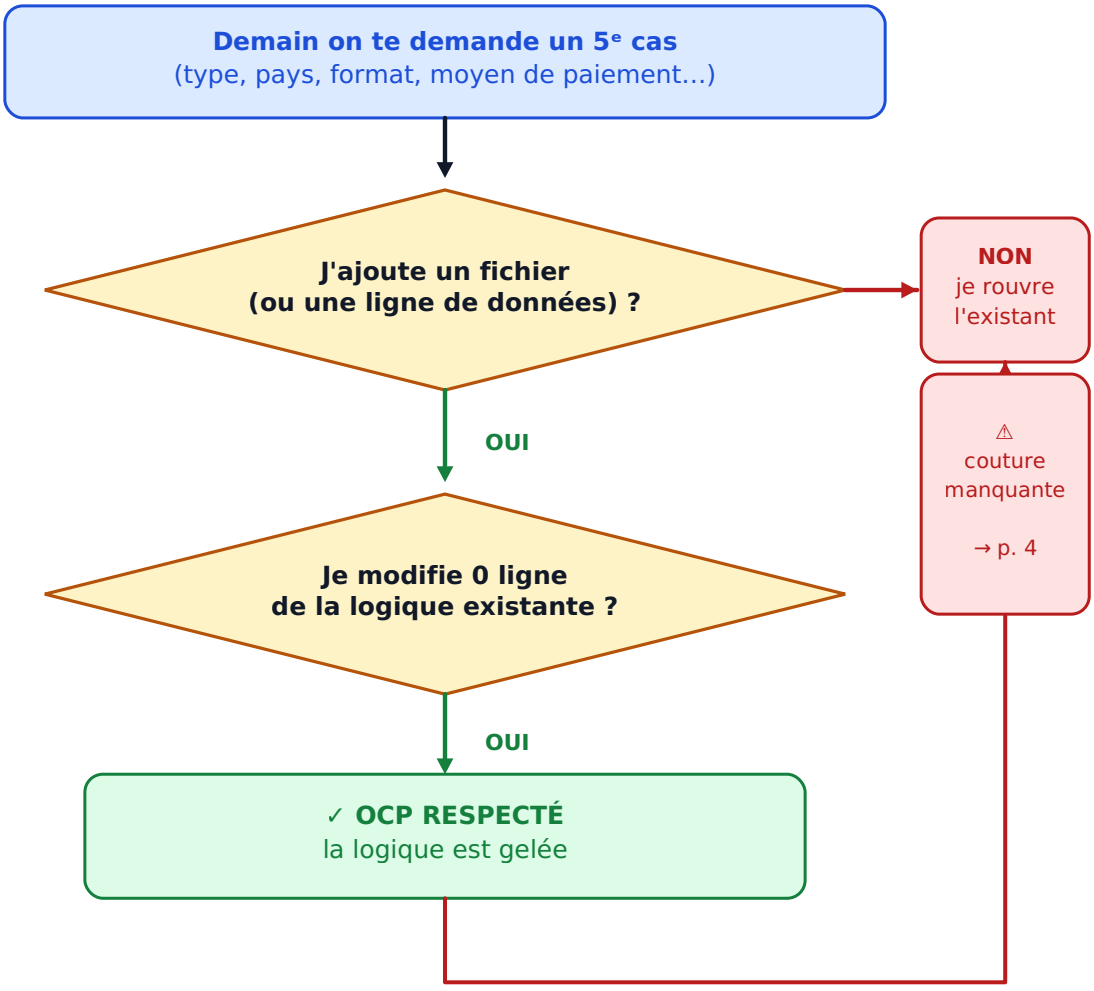
APRÈS — ouvert à l'extension

```
REMISES.push(remiseEtudiant);   // ← on ajoute ici
// calculerPrixFinal() : zéro ligne modifiée
// 1 seul fichier nouveau à tester
```

CE QUE L'OCP N'EST PAS	CE QUE L'OCP EST
« ne jamais modifier de code »	Ne jamais rouvrir la **logique**. Ajouter un fichier ou une ligne de registre est normal.
« une interface partout »	Une interface là où un **2ᵉ cas est réel**. Sinon elle coûte sans rien rapporter.
« un principe de classes »	Un principe de **frontières** : il fonctionne aussi avec des fonctions et des données.

Le test des 5 minutes

Deux questions à se poser devant chaque `if / else if` qui s'allonge.



A · Le vrai bénéfice

Ce n'est pas l'élégance : c'est le **périmètre de test**. Un fichier neuf = seuls ses tests sont à écrire.

B · Le coût caché de l'avant

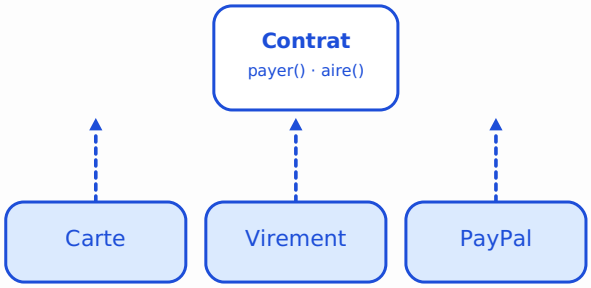
Ajouter le 5e cas oblige à re-valider les 4 précédents : **5 fois plus de surface** à vérifier.

LE SIGNAL	CE QU'IL RÉVÈLE	COUTURE	
1	Un 4e `else if` sur le même champ	l'axe de variation est le **type**	registre
2	La même condition recopiée dans 3 fichiers	il manque une **source de vérité**	données
3	Un case de 40 lignes	une variante = un vrai **algorithme**	interface
4	Des booléens en cascade dans l'orchestrateur	l'axe est la **suite de traitements**	pipeline
5	Un bug dans une branche oblige à relire les autres	aucune **frontière** n'existe	à coudre

Les 4 coutures, et rien d'autre

Pendre un point d'extension se fait toujours de l'une de ces quatre façons.

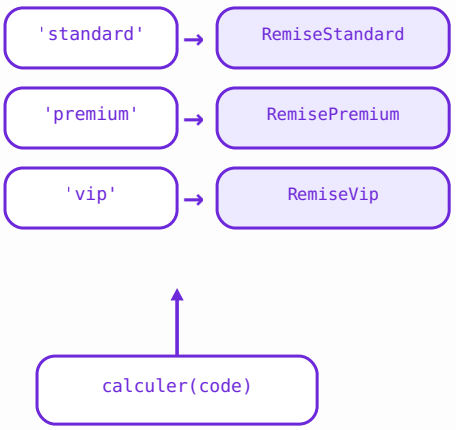
1 Interface + implémentations — polymorphisme



l'appelant ignore QUI exécute

quand : plusieurs façons de faire
un même geste métier

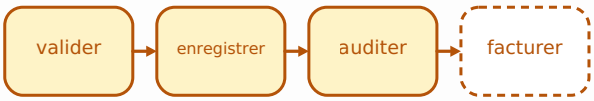
2 Registre — clé → module



nouvelle clé = 1 ligne

3 Liste d'étapes — pipeline

`for (étape of ETAPES) …`



l'étape en pointillés n'existait pas hier

l'orchestrateur ne bouge pas

quand : une suite de traitements
à exécuter dans l'ordre

4 Données / config — règles en tableau

pays · seuil · base · /kg



la logistique ajoute un pays
sans développeur

quand : ça varie en VALEUR
tarifs, seuils, quotas, taux

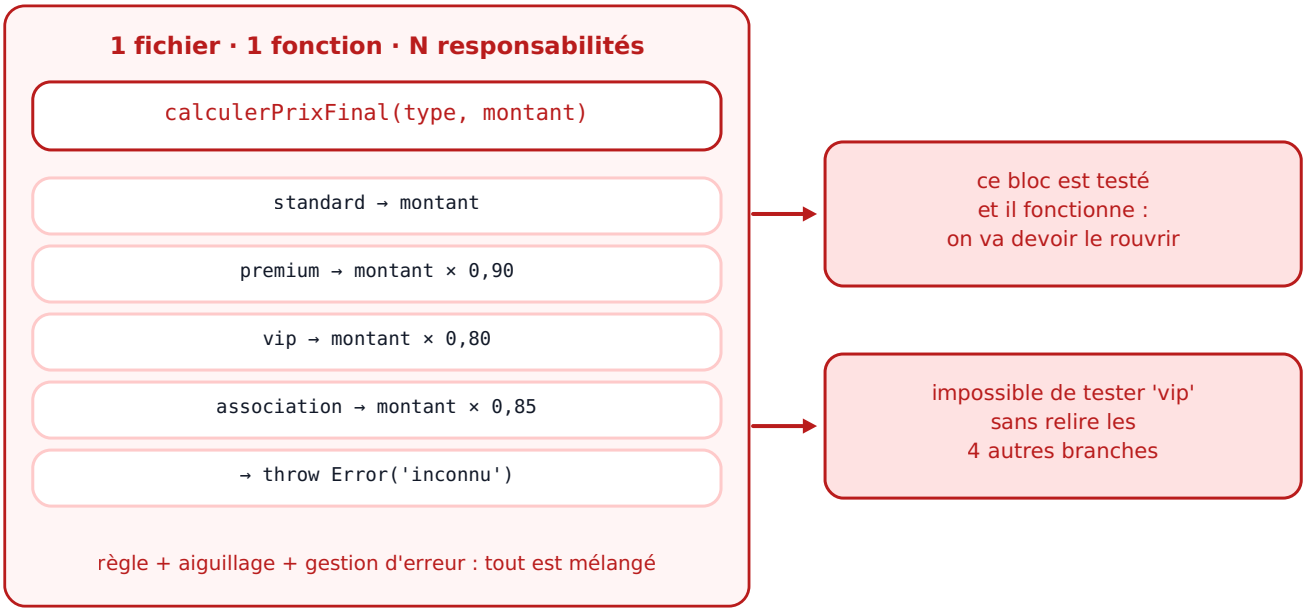
> **Erreur fréquente :** croire que l'OCP = forcément des classes et des interfaces. Pour des tarifs, des seuils, des quotas (couture 4), **une table de données est un OCP plus solide et plus léger** qu'une hiérarchie de classes.

LE SYMPTÔME DANS TON CODE	LA COUTURE	CE QUE TU AJOUTES DEMAIN
`if (type === …)` répété	**2 · Registre**	une entrée dans la Map + un fichier
`switch` sur un discriminant	**1 · Interface**	une classe qui implémente le contrat
`if (opt.audit) …`	**3 · Pipeline**	une étape dans le tableau
des nombres métier dans le code (4,90 · 20 %)	**4 · Données**	une ligne de config, voire rien (ops)

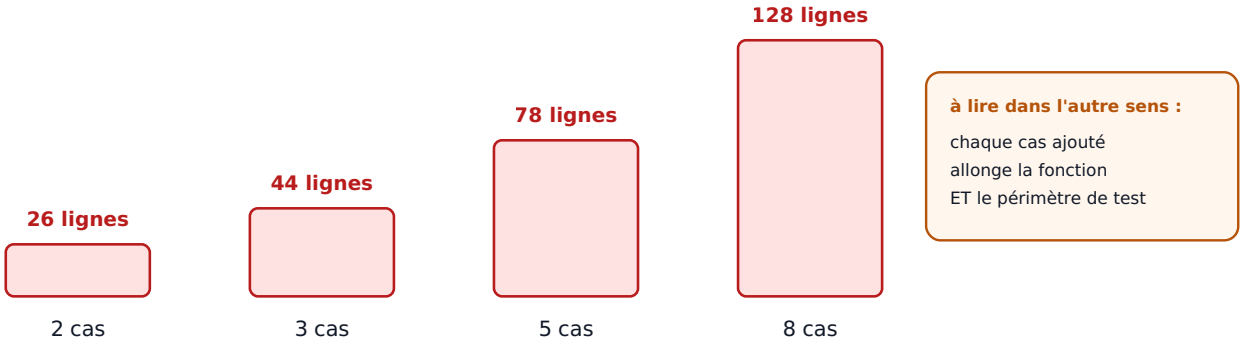
4 · LE DIAGNOSTIC

Anatomie de l'avant

À quoi ressemble le problème : un aiguillage qui héberge toutes les règles.



Ce qui grossit à chaque nouveau cas



le coût d'ajout est payé par du CODE MODIFIÉ

Le signal qui ne trompe pas : quand tu ajoutes un cas, tu dois **relire les autres cas** pour être sûr de ne rien casser. Le code est devenu une fondue de décisions et de règles.

A · Le coût invisible

Chaque ajout demande de **comprendre tout** le reste : la charge mentale grandit plus vite que le code.

B · Le coût de test

Une fonction de 80 lignes se teste par scénarios combinés ; un module de 20 lignes, unitairement.

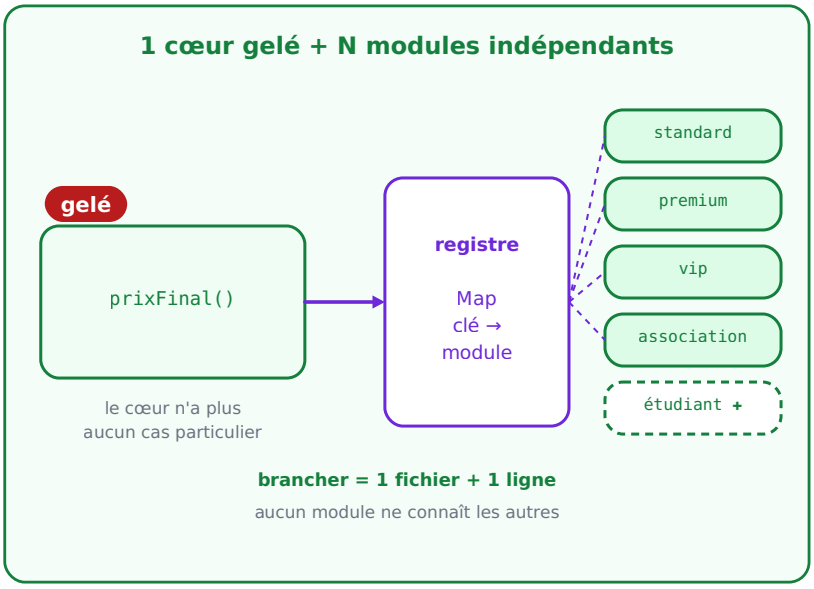
C · Le coût humain

Personne n'ose plus toucher. C'est le vrai plafond : le code fait **peur**.

5 · LA CIBLE

Anatomie de l'après

Le résultat visé : un cœur qui ne bouge plus, des modules qu'on branche.



Ce que voit la revue de code

AVANT — ajouter un cas

1 fichier modifié

≈ 40 lignes changées

APRÈS — ajouter un cas

+ 1 nouveau fichier

+ 1 ligne de registre

Coût du 5e cas

AVANT — relire + retester — ≈ 2 h

APRÈS — 1 module — ≈ 10 min

Zéro risque sur l'existant.

Le critère de réussite : pour le prochain cas, la revue de code affiche **1 nouveau fichier + 1 ligne d'enregistrement**, et zéro ligne rouge dans le cœur.

A · En revue

On relit **un module neuf**, pas un diff de 40 lignes au milieu d'une fonction connue.

B · En test

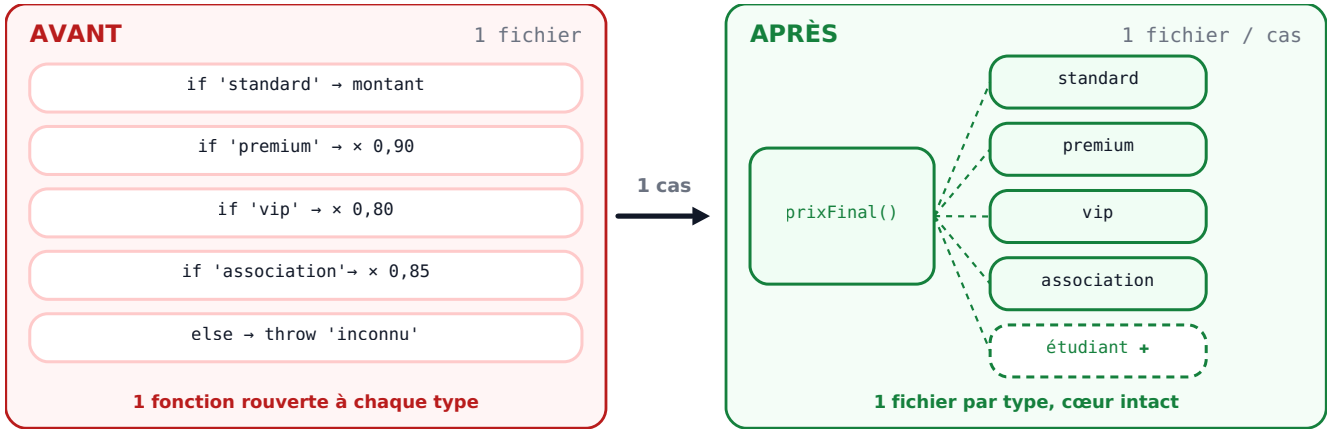
Chaque module a son test unitaire ; le cœur garde les siens, inchangés.

C · En équipe

Un nouveau peut livrer un cas **sans comprendre tout le système**. C'est ça, la vraie victoire.

1 Remises client

le if/else if sur un champ « type »



AVANT

```ts
// remises.ts
function prixFinal(type: string, m: number) {
  if (type === "standard")    return m;
  if (type === "premium")     return m * 0.9;
  if (type === "vip")         return m * 0.8;
  if (type === "association") return m * 0.85;
  throw new Error("type inconnu");
}
```

APRÈS

```ts
// remise.ts — un contrat, un registre
interface Remise {
  type: string; appliquer(m: number): number;
}

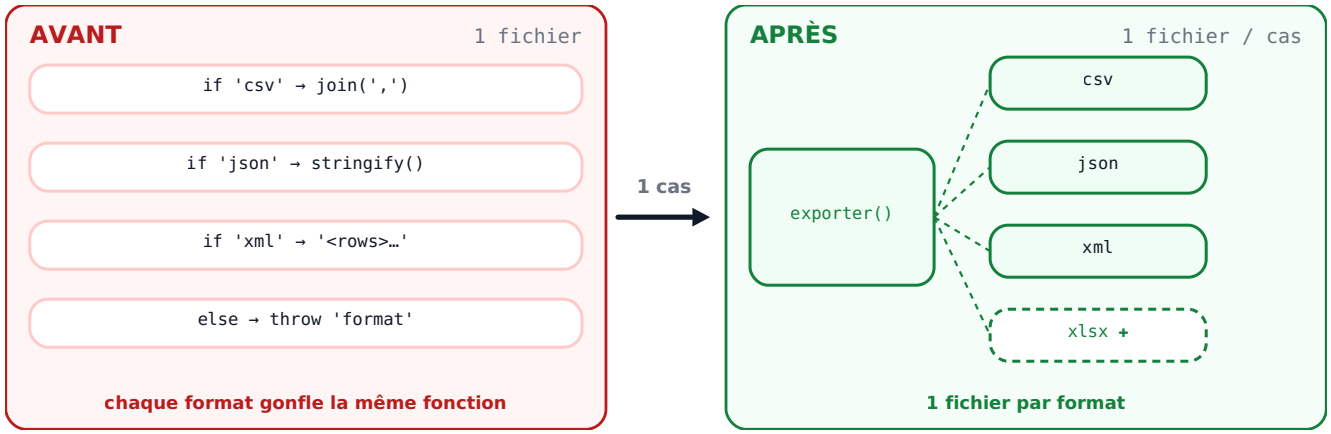
const REMISES: Remise[] = [standard, premium, vip, asso];
const parType = new Map(REMISES.map(r => [r.type, r]));

function prixFinal(type: string, m: number) {
  const r = parType.get(type);
  if (!r) throw new Error("type inconnu");
  return r.appliquer(m);  // ← inchangé à jamais
}
```

À retenir : la seule ligne qui reste « aiguillage » est le `get` du registre. Nouveau client = `REMISES.push()`, et le test du 5e cas ne concerne que le nouveau module.

2 Export de rapport

le if/else if sur un champ « format »



AVANT

```ts
// rapport.ts
function exporter(format: string, lignes: Ligne[]) {
  if (format === "csv")  return toCsv(lignes);
  if (format === "json") return JSON.stringify(lignes);
  if (format === "xml")  return toXml(lignes);
  throw new Error("format non supporté");
}
```

APRÈS

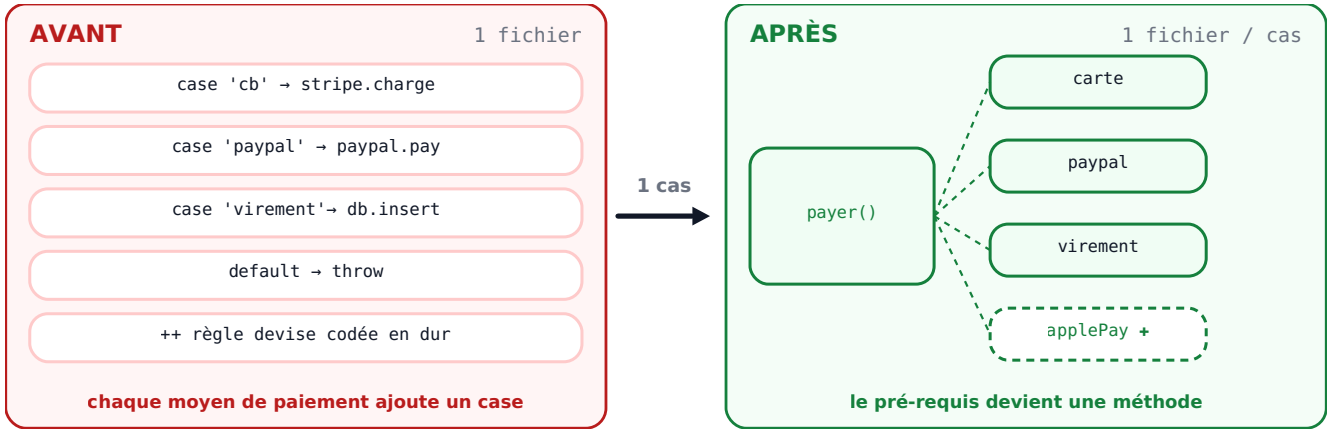
```ts
// exporter.ts
interface Exportateur {
  format: string; extension: string;
  exporter(l: Ligne[]): string;
}
const EXPORTATEURS = [csv, json, xml];
const parFormat = new Map(
  EXPORTATEURS.map(e => [e.format, e]));

function exporter(format: string, l: Ligne[]) {
  const e = parFormat.get(format);
  if (!e) throw new Error("format non supporté");
  return e.exporter(l);
}
```

À retenir : « un format = un fichier » rend l'export **testable isolément** : on vérifie le CSV sans monter un rapport entier, et le futur XLSX n'a aucun moyen de casser le XML.

3 Moyens de paiement

le switch sur un discriminant



AVANT

```ts
// paiement.ts
function payer(methode: string, m: number, dev: string) {
  switch (methode) {
    case "cb":
      if (dev !== "EUR") throw new Error("…");
      return stripe.charge(m, dev);
    case "paypal":   return paypal.pay(m, dev);
    case "virement": return db.insert({ m, dev });
    default: throw new Error("méthode inconnue");
  }
}
```

APRÈS

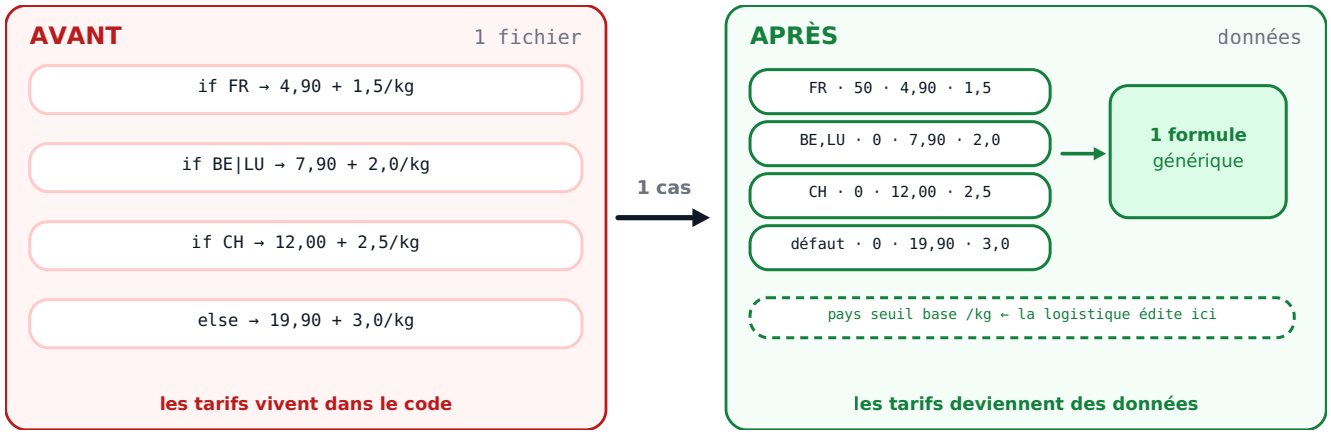
```ts
// moyen-paiement.ts
interface MoyenPaiement {
  code: string;
  supports(devise: string): boolean;   // ← pré-requis
  payer(montant: number, devise: string): Promise;
}
const MOYENS = [carte, paypal, virement];

function payer(code: string, m: number, dev: string) {
  const moyen = MOYENS.find(x => x.code === code);
  if (!moyen) throw new Error("méthode inconnue");
  if (!moyen.supports(dev)) throw new Error("devise");
  return moyen.payer(m, dev);
}
```

À retenir : les **règles de support** (« EUR seulement ») sortent du case et deviennent une méthode testable. Un case de 40 lignes ne se teste jamais seul ; une classe, si.

4 Frais de port

les nombres métier codés en dur



AVANT

```ts
// port.ts — chaque nouveau pays = 1 if de plus
function port(pays: string, kg: number, panier: number) {
  if (pays === "FR") {
    if (panier > 50) return 0;
    return 4.9 + Math.ceil(kg) * 1.5;
  }
  if (pays === "BE" || pays === "LU") return 7.9 + kg * 2;
  if (pays === "CH") return 12 + kg * 2.5;
  return 19.9 + kg * 3;
}
```

APRÈS

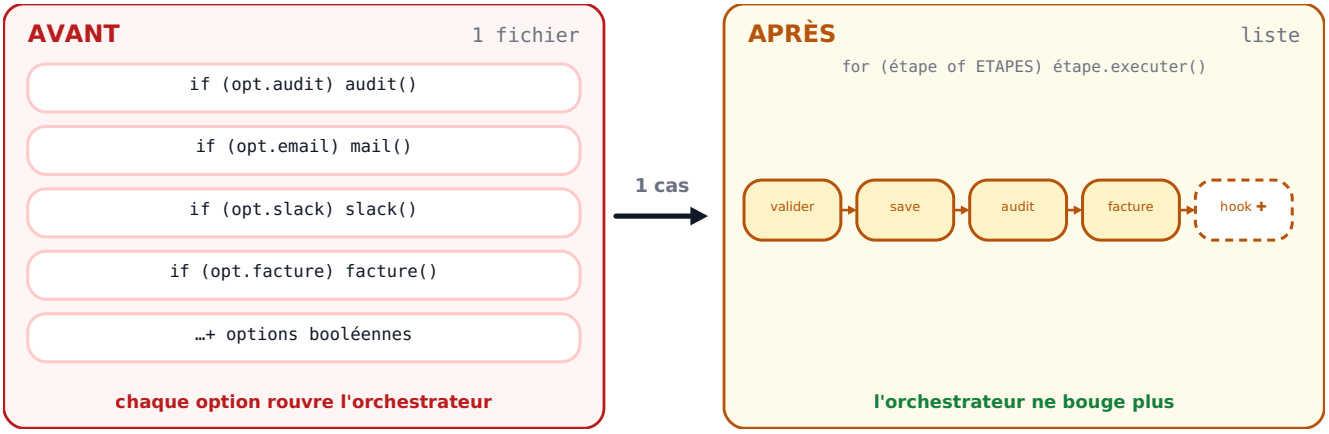
```ts
// port.ts — le CODE ne bouge plus
const REGLES = [
  { pays: ["FR"],      seuil: 50, base: 4.9, parKg: 1.5 },
  { pays: ["BE","LU"], seuil: 0,  base: 7.9, parKg: 2.0 },
  { pays: ["CH"],      seuil: 0,  base: 12,  parKg: 2.5 },
];
const DEFAUT = { pays: [], seuil: 0,
                 base: 19.9, parKg: 3 };

function port(pays: string, kg: number, panier: number) {
  const r = REGLES.find(x => x.pays.includes(pays))
            ?? DEFAUT;
  if (r.seuil > 0 && panier >= r.seuil) return 0;
  return r.base + Math.ceil(kg) * r.parKg;
}
```

À retenir : quand ça varie en **valeur** (tarifs, seuils, taux), la meilleure extension est **une ligne de données** — voire une ligne en base, modifiable sans redéploiement. Aucune classe nécessaire.

5 Traitement de commande

les options booléennes en cascade



AVANT

```
// commande.ts
async function traiter(cmd: Commande, opt: Options) {
  await valider(cmd);
  await enregistrer(cmd);
  if (opt.audit)   await ecrireAudit(cmd);
  if (opt.email)   await envoyerEmail(cmd);
  if (opt.slack)   await notifierSlack(cmd);
  if (opt.facture) await facturer(cmd);   // ajouté hier
}
```

APRÈS

```
// commande.ts
interface Etape {
  nom: string; executer(c: Commande): Promise;
}

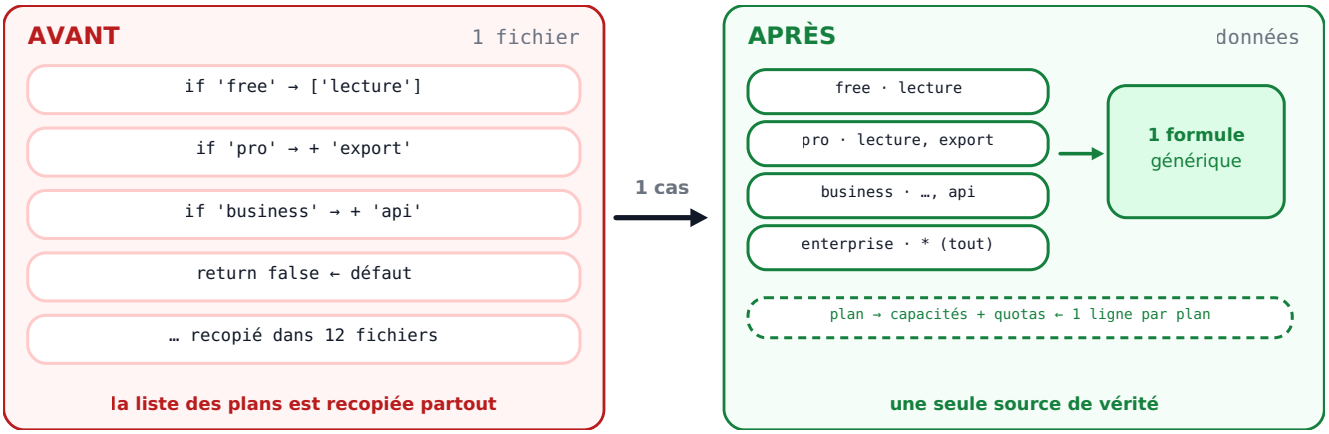
const ETAPES: Etape[] = [
  etapeValidation, etapePersistance,
  etapeAudit, etapeEmail, etapeFacture,
];

async function traiter(cmd: Commande) {
  for (const e of ETAPES) await e.executer(cmd);
}
// nouvelle étape = nouveau fichier + 1 ligne ci-dessus
```

À retenir : une **suite de traitements** se modélise en liste, pas en cascade de booléens. Bonus : on peut réordonner, désactiver ou tester une étape en changeant simplement le tableau.

6 Plans d'abonnement

la même condition recopiée partout



AVANT

```
// plan.ts — et la même liste dans 12 fichiers…
function peutUtiliser(plan: string, f: string): boolean {
  const pro = ["lecture", "export"];
  if (plan === "free")       return f === "lecture";
  if (plan === "pro")        return pro.includes(f);
  if (plan === "business")   return f !== "sso";
  if (plan === "enterprise") return true;
  return false;
}
```

APRÈS

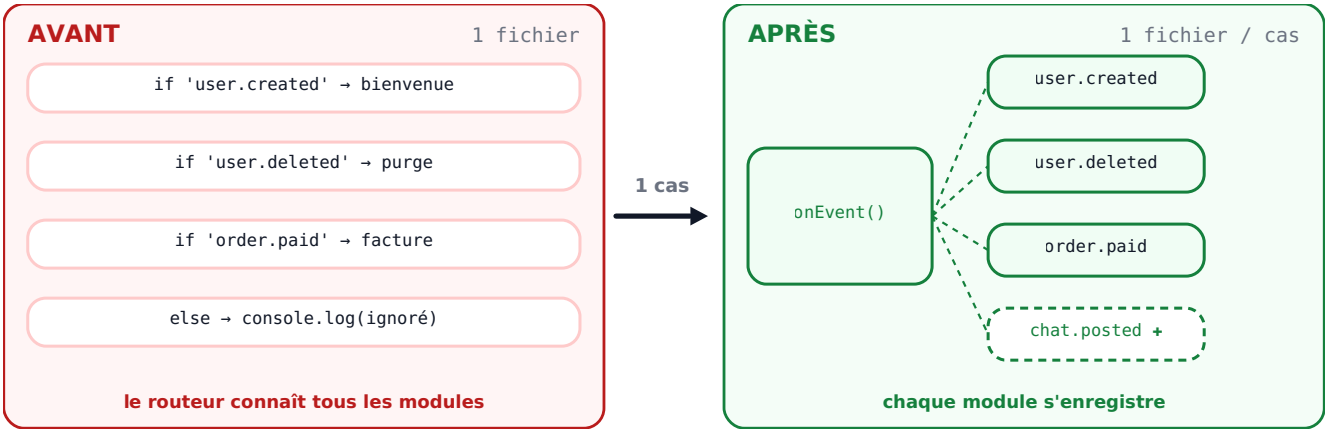
```
// plan.ts — la seule source de vérité
const PLANS = [
  { code: "free",    cap: ["read"],           sieges: 1 },
  { code: "pro",     cap: ["read","export"], sieges: 5 },
  { code: "business", cap: ["read","export","api"] },
  { code: "enterprise", cap: ["*"] },
];
const plan = (c: string) => PLANS.find(p => p.code === c);

function peutUtiliser(c: string, f: string) {
  const p = plan(c);
  if (!p) return false;
  return p.cap.includes("*") || p.cap.includes(f);
}
```

À retenir : quand une règle est **recopiée** dans plusieurs fichiers, l'axe de variation est la donnée elle-même. Une capacité (cap) rend la question « ce plan peut-il… ? » unique dans toute l'app.

7 Événements & handlers

le if/else if sur le type d'événement



AVANT

```
// events.ts — le fichier que tous les modules modifient
async function onEvent(e: { type: string; data: any }) {
  if (e.type === "user.created") await bienvenue(e.data);
  else if (e.type === "user.deleted") await purge(e.data);
  else if (e.type === "order.paid") await facturer(e.data);
  else console.log("event ignoré", e.type);
}
```

APRÈS

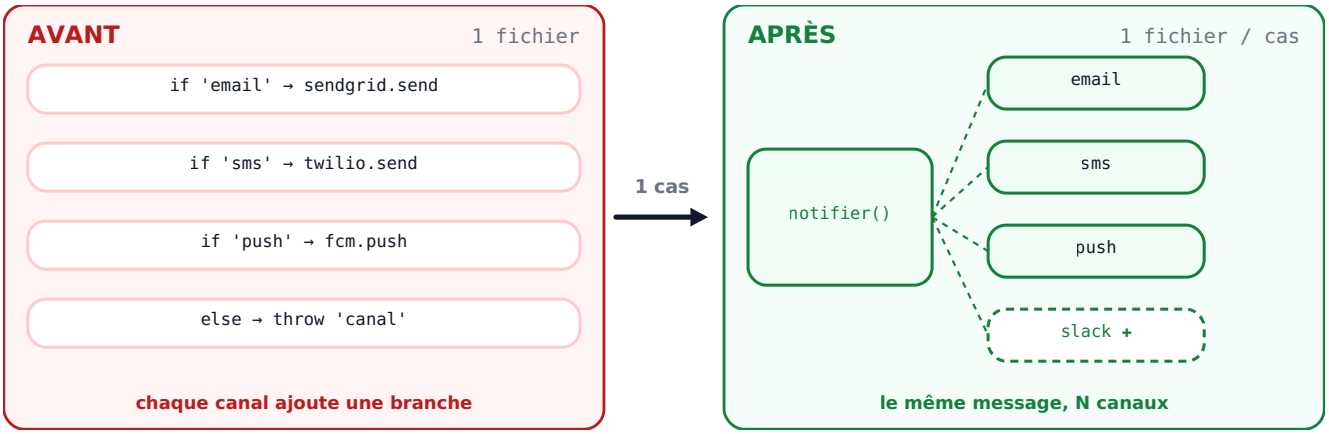
```
// events.ts — le bus
const HANDLERS = new Map();
export const on = (t: string, h: Handler) =>
  HANDLERS.set(t, [...(HANDLERS.get(t) ?? []), h]);

async function onEvent(e: { type: string; data: any }) {
  const hs = HANDLERS.get(e.type) ?? [];
  await Promise.all(hs.map(h => h(e.data)));
}
// user.ts (autre fichier)
on("user.created", envoyerBienvenue);
```

À retenir : le bus d'événements est l'OCP au niveau de l'**application** : ajouter un module = l'importer, sans jamais toucher au routeur. C'est le mécanisme derrière les plugins et les `middlewares`.

8 Notifications

le if/else if sur le canal



AVANT

```
// notifier.ts
async function notifier(u: User, c: string, msg: string) {
  if (c === "email") await sendgrid.send(u.email, msg);
  else if (c === "sms") await twilio.send(u.phone, msg);
  else if (c === "push") await fcm.push(u.device, msg);
  else throw new Error("canal inconnu");
}
```

APRÈS

```
// canal.ts
interface Canal {
  code: string;
  envoyer(u: User, msg: string): Promise;
}
const CANAUX = [canalEmail, canalSms, canalPush];
const parCode = new Map(CANAUX.map(c => [c.code, c]));

function notifier(u: User, codes: string[], m: string) {
  const canaux = codes.map(c => parCode.get(c)!);
  return Promise.all(canaux.map(c => c.envoyer(u, m)));
  // "slack" ? un fichier de 10 lignes, et c'est tout
}
```

À retenir : l'interface permet aussi de passer d'**un** canal à **plusieurs** sans changer l'appel : c'est le signe qu'on a trouvé le bon axe de variation.

6 · TOUR D'HORIZON

Le reste de la boîte, en un coup d'œil

Six situations que tu croiseras en vrai : à gauche le symptôme, à droite la couture.

Formes géométriques
aire(), perimetre()
	`if 'cercle' → π·r²`
	`if 'rectangle' → l·h`
	`if 'triangle' → b·h/2`
		→ **1 fichier / cas**
		polymorphisme
		contrat : aire()
			quand :
			un même geste, plusieurs algos

Import de fichiers
csv, xml, xls…
	`if mime === 'csv' → …`
	`if mime === 'xml' → …`
	`throw 'non géré'`
		→ **1 fichier / cas**
		interface
		le critère : accepte()
			quand :
			quand le test n'est pas une égalité

TVA par pays
taux qui changent
	`if FR → 20 % / 5,5 %`
	`if DE → 19 % / 7 %`
	`if CH → 8,1 %`
		→ **1 fichier / cas**
		données
		taux en config datée
			quand :
			quand ça varie chaque année

Armes / items de jeu
stats & effets
	`if 'epee' → 12`
	`if 'hache' → 18`
	`if 'arc' → 9 + saigne`
		→ **1 fichier / cas**
		données + effets
		contenu en JSON
			quand :
			le game designer ajoute sans dev

Étapes d'un import
parse → valide → save
	`parse()`
	`if (opt.valider) valider()`
	`if (opt.dedupe) dedupe()`
		→ **1 fichier / cas**
		liste d'étapes
		ETAPES = […]
			quand :
			quand c'est une suite de traitements

Cache / middlewares
autour d'un appel
	`if (opt.cache) …`
	`if (opt.log) …`
	`if (opt.retry) …`
		→ **1 fichier / cas**
		liste d'étapes
		wrappers chaînés
			quand :
			quand on empile des comportements autour

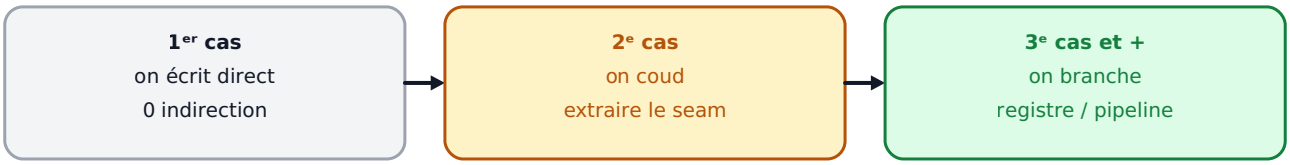
Comment choisir vite : regarde d'abord **ce qui change** (la valeur ? la clé ? la suite d'étapes ? l'algorithme ?). C'est ça qui désigne la couture — pas ton langage, ni le framework du moment.

7 · LE PIÈGE

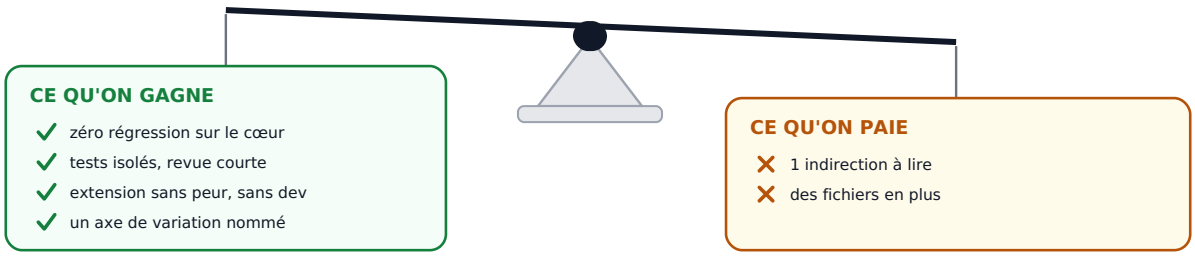
Sur-abstraire est aussi une faute

L'OCP a un prix : une indirection. On ne la paie que si le 2e cas est réel.

La règle de 2 : on ne coud pas avant d'avoir le 2e cas



Ce que l'OCP échange



rentable dès le 2e cas réel — ruineux si le 2e cas n'arrive jamais

✗ Sur-abstraction (à éviter)

```
interface RemiseStrategyFactoryProvider
abstract class AbstractRemiseStrategy
```

...pour 1 seule remise, 0 deuxième prévue.

Le remède est pire que le mal.

✓ YAGNI assumé

```
const REMISE_PREMIUM = 0.9;
const prixPremium = (m) => m * REMISE_PREMIUM;
```

Lisible, testable, 1 ligne.

On coudra quand le 2e cas arrivera.

1 · Le 2e cas existe ?

Alors couds : l'indirection est déjà **rentabilisée**.

2 · Il est au backlog ?

Couds aussi : c'est un investissement, pas une fiction.

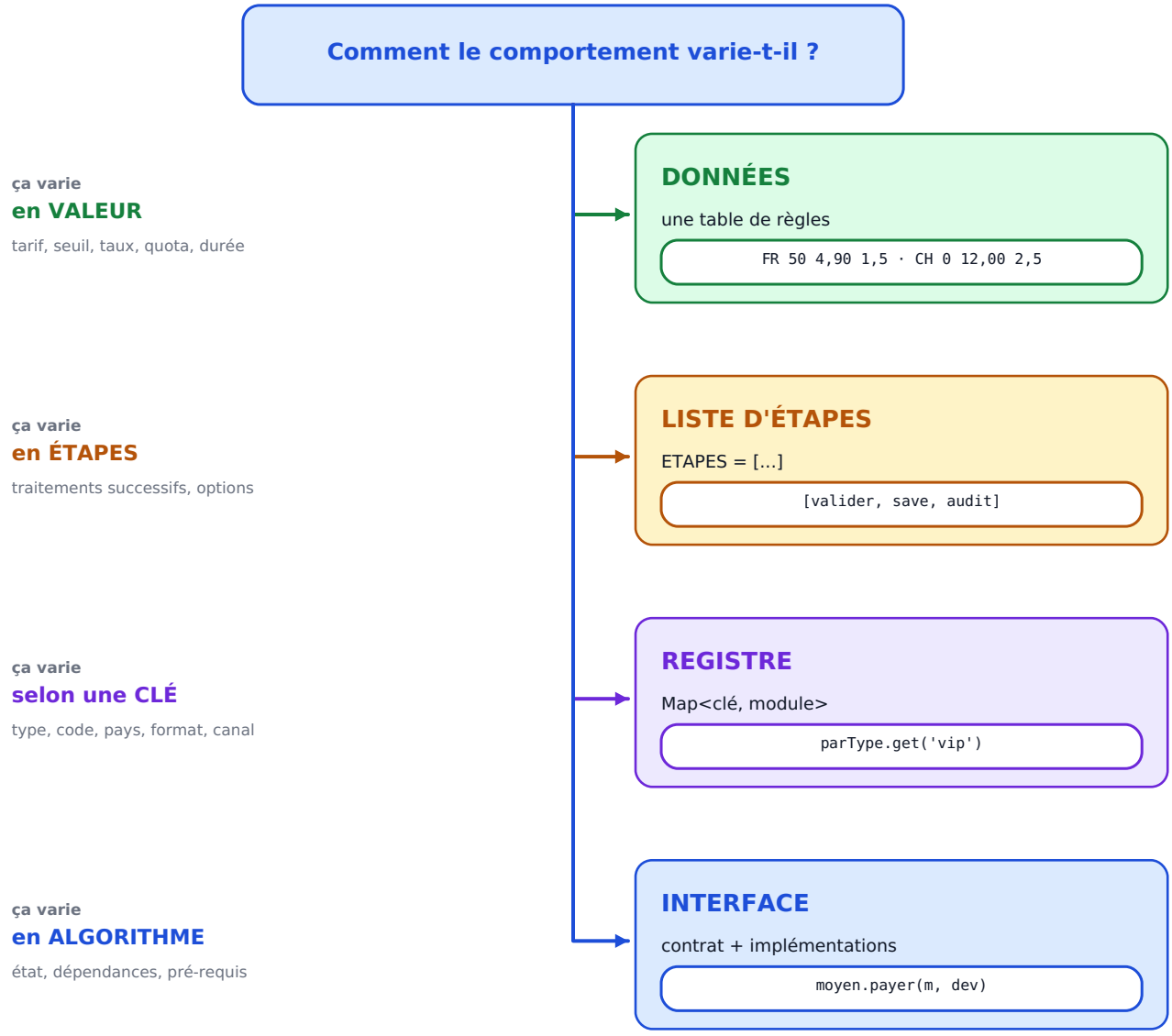
3 · C'est une hypothèse ?

Écris direct, simplement. Tu coudras au vrai 2e cas.

LES 4 ERREURS DU DÉBUTANT	LE BON GESTE
Abstraire dès le 1er cas	Écrire direct. Coudre au **2e cas réel**.
Mettre une interface sur chaque classe	Une interface = un **axe de variation** nommé, pas un réflexe.
Croire qu'on ne doit plus jamais modifier	Ajouter une ligne de registre **est** de l'extension.
Laisser dormir une couture jamais utilisée	Un seam sans 2e cas est du **code mort** : on le supprime.

Quelle couture pour ce cas-là ?

Une seule question : de quelle **nature** est la variation ?



Un seul cas, aucune deuxième variante prévue → on écrit direct.

Le seam s'ajoute au 2e cas réel, pas avant. (YAGNI)

A · Deux axes se croisent

Sépare : **une couture par axe** (ex. canal de notif × format de message).

B · Ça se compose autour d'un appel

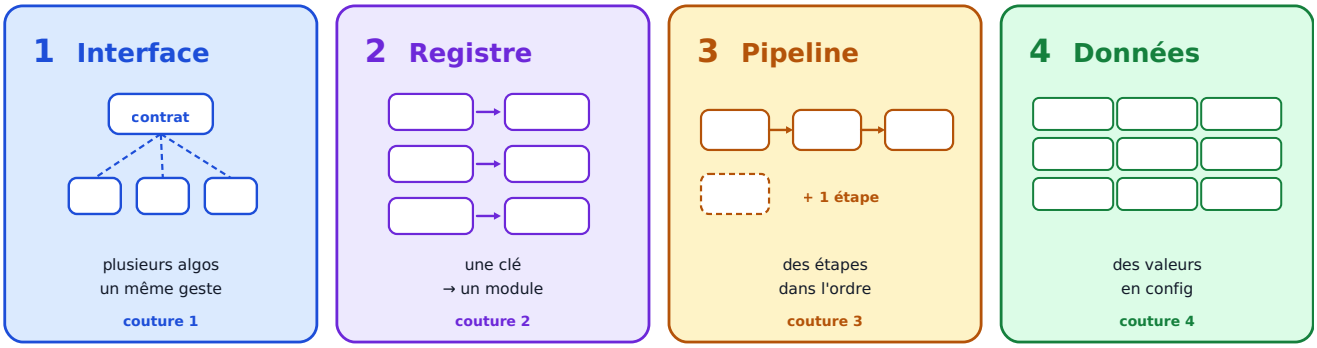
Alors **pipeline / wrappers** plutôt qu'interface : cache, log, retry.

C · Aucune 2e variante en vue

Écris direct et simplement. La couture viendra au 2e cas réel.

Récap visuel et cartes de révision

Si tu ne dois retenir qu'une page, c'est celle-ci.



En une phrase ?
Ouvert à l'extension, fermé à la modification : le cœur testé ne bouge plus, on branche à côté.

Le test ?
Pour le 5e cas, j'ajoute un fichier ou je rouvre une fonction testée ?

Les 4 coutures ?
Interface · Registre (Map) · Liste d'étapes · Données.

Le symptôme ?
if/else if sur un champ, ou la même condition recopiée dans plusieurs fichiers.

Le contre-poids ?
YAGNI + règle de 2 : pas d'abstraction avant le 2e cas réel.

La preuve que c'est bon ?
La revue affiche 1 fichier + 1 ligne, et 0 ligne rouge dans le cœur.

1 · Sur ton code
Ouvre ta fonction la plus modifiée. Compte les `else if` : c'est ta dette.

2 · Sur tes données
Cherche les nombres métier dans le code (4,90 · 20 % · 50). Ils veulent sortir.

3 · Sur ta PR
Ce que la revue verra au prochain cas : 1 fichier neuf, ou 40 lignes rouges ?

CHECKLIST AVANT DE POSER UNE COUTURE	SI NON →
Le nouveau cas a-t-il **son propre fichier** ?	la logique est encore mélangée
Le cœur garde-t-il **zéro** cas particulier ?	ce n'est pas une couture, c'est un déplacement
Le registre / la liste est-il **le seul point** à éditer ?	il y a plusieurs sources de vérité
Chaque module est-il **testable seul** ?	la frontière est mal placée

Et si tu oublies tout le reste : le jour où tu écris un `if` qui ajoute un cas au bout d'une chaîne de quatre, pose-toi la question du seam. C'est tout l'OCP.